Cryptography

University of Insubria

Number Theory

Number Sets

- N: Natural Numbers
  - {0, 1, 2, ..}
- Z: Integers
  - {.., −2, −1, 0, 1, 2, ..}
- Z^+: Positive Integers
  - {1, 2, ..}

Number Theory

Integers mod N

- for N in Z
  - $Z_N = \{0, 1, .., N - 1\}$
- $Z^*_N = \{v \in Z_N : gcd(a, N) = 1\}$

Example

- N = 12
- Z_{12} = ?
- Z^*_{12} = ?

Number Theory

Integers mod N

- for N in Z
  - $Z_N = \{0, 1, .., N - 1\}$
- $Z^*_N = \{v \in Z_N : gcd(a, N) = 1\}$

Example

- N = 12
- $Z_{12} = \{0, 1, 2, .., 11\}$
- $Z^*_{12} = ?$

Number Theory

Integers mod N

- for N in Z
  - $Z_N = \{0, 1, .., N - 1\}$
- $Z^*_N = \{v \in Z_N : gcd(a, N) = 1\}$

Example

- N = 12
- $Z_{12} = \{0, 1, 2, .., 11\}$
- $Z^*_{12} = \{1, 5, 7, 11\}$

- N = 13
- $Z_{13} = \{0, 1, 2, .., 12\}$
- $Z^*_{13} = \{1, 2, .., 12\}$

Number Theory

Division

- div(a, N) = (q, r)
  - q: quotient
  - r: reminder

Modulus

- a mod N = r $\in Z_N$

Example

- div(17,3) = ?
- div(14,3) = ?

- 17 mod 3 = ?
- 14 mod 3 = ?

Number Theory

Division

- div(a, N) = (q, r)
  - q: quotient
  - r: reminder

Modulus

- a mod N = r $\in Z_N$

Example

- div(17,3) = (5,2)
- div(14,3) =?

- 17 mod 3 = ?
- 14 mod 3 = ?

Number Theory

Division

- div(a, N) = (q, r)
  - q: quotient
  - r: reminder

Modulus

- a mod N = r $\in Z_N$

Example

- div(17,3) = (5,2)
- div(14,3) = (4,2)

- 17 mod 3 = ?
- 14 mod 3 = ?

Number Theory

Division

- div(a, N) = (q, r)
  - q: quotient
  - r: reminder

Modulus

- a mod N = r $\in Z_N$

Example

- div(17,3) = (5,2)
- div(14,3) = (4,2)

- 17 mod 3 = 2
- 14 mod 3 = 2

Number Theory

Division

- div(a, N) = (q, r)
  - q: quotient
  - r: reminder

Modulus

- a mod N = r $\in Z_N$
  - (a + b) mod N = (a mod N + b mod N) mod N
  - (a * b) mod N = (a mod N * b mod N) mod N

Example

- div(17,3) = (5,2)
- div(14,3) = (4,2)

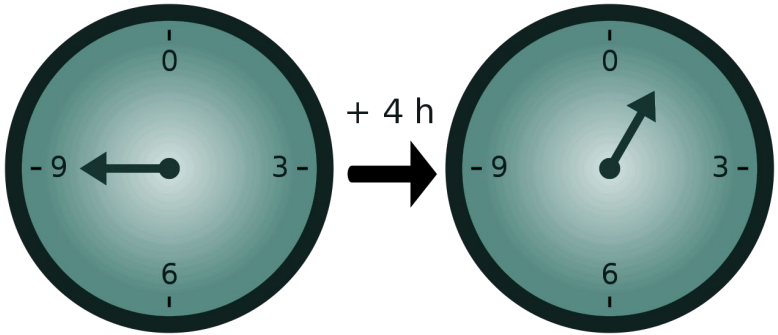
- 17 mod 3 = 2
- 14 mod 3 = 2

- (7 + 4) mod 3 = 11 mod 3 = 2
- (7 mod 3 + 4 mod 3) mod 3 = (1 + 1) mod 3 = 2

Modular Congruence

Given three integers a, b, n we say that a is congruent to b modulo n ($a \equiv b \bmod n$) if

- $a - b$ is divisible by n
- a and b give the same remainder if divided by n



Properties

- $a \equiv a \bmod n$
- $a \equiv b \bmod n \Rightarrow b \equiv a \bmod n$
- $a \equiv b \bmod n$ e $b \equiv c \bmod n \Rightarrow a \equiv c \bmod n$

Modular Congruence

Example

- $32 \equiv 7 \bmod 5$
  - $32 - 7 = 25$ is divisible by 5 (25/5 = 5, r=0)
    - 32 / 5 = 6, r= 2
    - 7 / 5 = 1, r=2
- $91 \not\equiv 18 \bmod 3$
  - $91 - 18 = 73$ is not divisible by 3 (73 / 3 = 24, r=1)
    - 91 / 3 = 30, r=1
    - 18 / 3 = 6, r= 0

Modular Congruence

Properties

- Reflexivity: $a \equiv a \pmod{n}$
- Symmetry: $a \equiv b \pmod{n} \Rightarrow b \equiv a \pmod{n}$
- Transitivity: $a \equiv b \pmod{n}$ e $b \equiv c \pmod{n} \Rightarrow a \equiv c \pmod{n}$

Additional properties

- If $a \equiv a'$ (mod N) and $b \equiv b'$ (mod N)
  - $a + b \equiv a' + b'$ (mod N)
  - $ab \equiv a'b'$ (mod N)
  - $ka \equiv ka'$ (mod N) for any integer k

Number Theory

Division

- div(a, N) = (q, r)
  - q: quotient
  - r: reminder

Modulus

- a mod N = r $\in Z_N$

Modular Congruence

- a ≡ b mod N if a mod N = b mod N

Example

- div(17,3) = (5,2)
- div(14,3) = (4,2)

- 17 mod 3 = 2
- 14 mod 3 = 2

- 17 ≡ ?

Number Theory

Division

- div(a, N) = (q, r)
  - q: quotient
  - r: reminder

Modulus

- a mod N = r $\in Z_N$

Modular Congruence

- a ≡ b mod N if a mod N = b mod N

Example

- div(17,3) = (5,2)
- div(14,3) = (4,2)

- 17 mod 3 = 2
- 14 mod 3 = 2

- 17 ≡ 14 mod 3

Code

Euclidean Algorithm

Euclidean Algorithm

GCD - Greatest Common Divisor

- the gcd of two or more integers is the largest positive integer that divides each of the integers
  - given two integers x,y, the gcd is denoted gcd(x, y)

Euclidean Algorithm

- an efficient way to compute the gcd of two integers

```
a, b integers (a ≥ b)
A <- a, B <- b
while B != 0:
    new *
    R <- A mod B
    A <- B, B<-R
return A
```

Euclidean Algorithm

```python
def gcd(a, b):
    a, b = abs(a), abs(b)
    while a > 0:
        b = b % a
        tmp = a; a = b; b = tmp
    return b

print(gcd(126, 147))  # 21
```

```python
def gcd(a, b):
    while b > 0:
        r = a % b
        a, b = b, r
    return a

print(gcd(126, 147))  # 21
```

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Challenge

Crypto 08 - Congruenze Modulari

Crypto 08 - Congruenze Modulari

Buondì! Rispondi correttamente a questi quesiti sulle congruenze modulari per ottenere la flag.

- -57 % 86 = ?
  - print(-57 % 86) # 29
- -5 % 80 = ?
  - print(-5 % 80) # 75
- 782 == 616 (mod 83)? (si/no)
  - print((782-616)/83) # si
- 870 == 215 (mod 90)? (si/no)
  - print((870-215)/90) # no

flag: flag{t1ck_t0ck_M4th_1s_0n_th3_Cl0cK}

Group

A (finite) group is a (finite) non-empty set G equipped with a binary operation * s.t. the following properties hold

- Closure
  - $\forall\ g, h \in G, g * h \in G$
- Identity:
  - $\exists\ e \in G$ s.t. $\forall\ g \in G, g * e = e * g = g$
- Inverse:
  - $\forall\ g \in G\ \exists\ h \in G$ s.t. $g * h = e = h * g$
- Associativity:
  - $\forall\ g, h, f \in G, (g * h) * f = g * (h * f)$

A group is abelian if $\forall\ g, h \in G, g * h = h * g$

Group

Example of groups

- $(Z, +)$
- $(Q, +)$
- $(Q, *)$
- $(Z_N, +)$

Not a group

- $(Z, *)$ -> no multiplicative inverses
- $(Z_N, *)$

Modular inverses

In Q the inverse of 5 is ⅕, but in Z_N?

- the inverse of x in Z_N is an element $y \in Z_N$ s.t. $x * y = 1$ in Z_N
- we denote y as x^{-1}

Lemma

- x in Z_N has inverse iff gcd(x, N) = 1

Bézout's Theorem

Theorem

- given a e n con GCD(a, N) = 1, esiste un unico intero b tale che $ab \equiv 1 \pmod{n}$

Indichiamo questo intero con a^{-1} e lo chiamiamo **inverso moltiplicativo** di a

Esempio

- $2^{-1} \pmod 5 = 3$, perche' $2 \cdot 3 = 6 \equiv 1 \pmod 5$

Bézout's Identity

The Bachet-Bézout identity is defined as

- if a and b are two integers and d is their GCD, then there exist u and v, two integers such that a * u + b * v = d

Example

- a=12, b=30, gcd (12,30)=6
  - $12 \times -2 + 30 \times 1 = 6$

Extended Euclidean Algorithm

The Extended GCD is an extension of the GCD to compute the GCD and the Bézout's identity

```python
def exgcd(a, b):
    if a == 0:
        return (b, 0, 1)
    else:
        b_div_a, b_mod_a = divmod(b, a)
        g, x, y = exgcd(b_mod_a, a)
        return g, y - b_div_a * x, x


print(exgcd(126, 147))  # 21, -1, 1
print(exgcd(25, 5))  # 5, 0, 1
```

Extended Euclidean Algorithm

The extended Euclidean algorithm allows to calculate the modular inverse

```python
def modinv(a, b):
    g, x, _ = exgcd(a, b)
    if g != 1:
        raise Exception('gcd(a, b) != 1')
    return x % b

print(modinv(67, 194))  # 139
```

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Extended Euclidean Algorithm

The extended Euclidean algorithm allows to calculate the modular inverse

Da Python 3.8

```
1  inv = pow(a, -1, b)
```

```
, b)
ion('gcd(a, b) != 1')
4))  # 139
```

dCode
https://www.dcode.fr/

Challenge

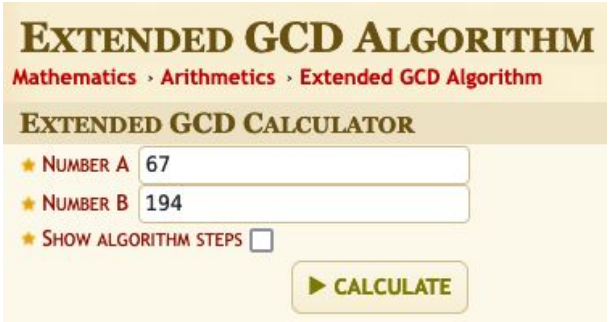
Crypto 09 - Inverso Modulare

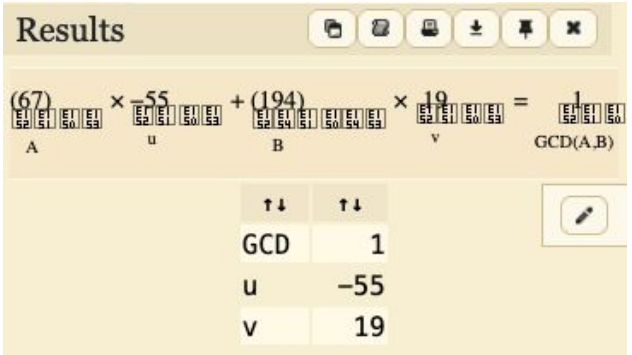
UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 09 - Inverso Modulare

Buondì! Rispondi correttamente a questi quesiti sugli inversi modulari per ottenere la flag.

- a = 67, b = 194, trova x, y tali che x * a + y * b == GCD(a, b)



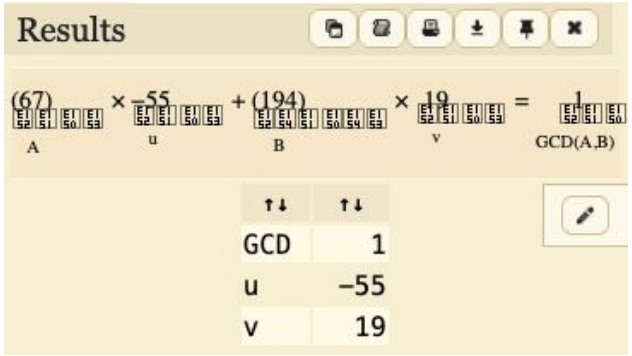


UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 09 - Inverso Modulare

Buondì! Rispondi correttamente a questi quesiti sugli inversi modulari per ottenere la flag.

- 67 è invertibile mod 194? (si/no)
  - si

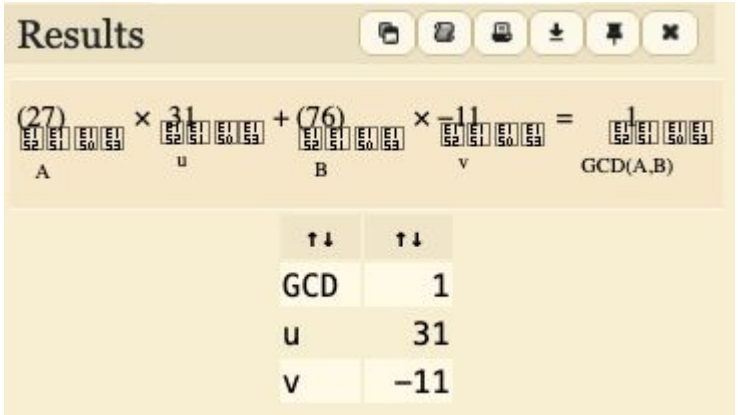


UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 09 - Inverso Modulare

Buondì! Rispondi correttamente a questi quesiti sugli inversi modulari per ottenere la flag.

- Qual è l'inverso di 27 modulo 76?
  - print(31 % 76) # 31



flag:flag{m3an1Ng_is_4l1_4b0uT_C0nT3xt}

Exponentiation

- $g^m = g * g * .. * g$, for $m \in G$
- $g^{-m} = g^{-1} * g^{-1} * .. * g^{-1}$, for $m \in G$

Properties

- $g^{i+j} = g^i * g^j$
- $g^{ij} = (g^i)^j = (g^j)^i$
- $g^{-i} = (g^i)^{-1} = (g^{-1})^i$

Example

$G = Z^*_{14}$

- $5^3 =$
- $= 5 * 5 * 5$
- $= 25 * 5$
- $\equiv 11 * 5$ # = 55
- $\equiv 13$

Order of a group

The order of a group G, denoted as $|G|$, is the number of elements in the group

- $|G| = m$

Example

- $G = Z^*_{14}$
  - $|G| = 18$

Cyclic groups

A cyclic group is a group that can be generated by a single element, called a generator

- Let G a finite group of order m and g an element of G
  - the order of g is the smallest integer i for which $g^i = 1$

- if i = m the g is a generator of G and G is called cyclic group

// If p is prime then Z^*_p is a cyclic group of order p-1

Useful kinds of primes

Strong prime

- p is large enough to be unfeasible to compute the factorisation of two primes having the size of p

Safe prime

- p = 2q + 1, q prime

// A large safe prime is probably (not always) a strong prime

Coprime Integers

Two integers a and b are **coprime** if the only positive integer that is a divisor of both is 1

- equivalent to GCD(a, b) = 1
- a is prime to b or a is coprime to b

Funzione di Eulero

We define $\varphi(n)$ as the number of integers k between 1 and n such that GCD(k, n) = 1

Euler's Theorem

- given a e N with GCD(a, N) = 1, then $a^{\varphi(n)} \equiv 1 \pmod{n}$

Properties

- $\varphi(1) = 1$
- $\varphi(p) = p - 1$ if p is a prime number
- $\varphi(pk) = p - 1\ p(k-1)$ if p is a prime number
- $\varphi(pq) = (p - 1)(q - 1)$ if p and q are prime number and p != q

Challenge

Crypto 10 - CRT

Crypto 10 - CRT

Buondì! Risolvi questo sistema di equazioni modulari per ottenere la flag.

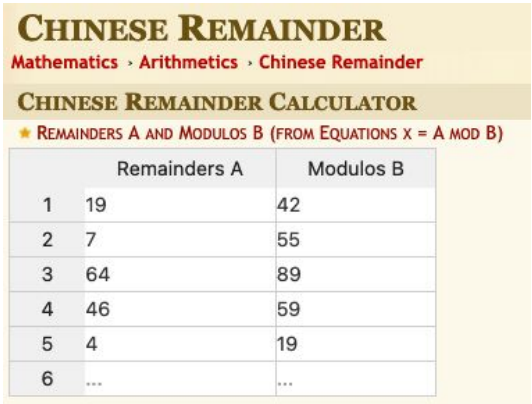
- x % 42 = 19
- x % 55 = 7
- x % 89 = 64
- x % 59 = 46
- x % 19 = 4
- x % 230466390 = ?

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 10 - CRT

Buondì! Risolvi questo sistema di equazioni modulari per ottenere la flag.

- x % 42 = 19
- x % 55 = 7
- x % 89 = 64
- x % 59 = 46
- x % 19 = 4
- x % 230466390 = ?



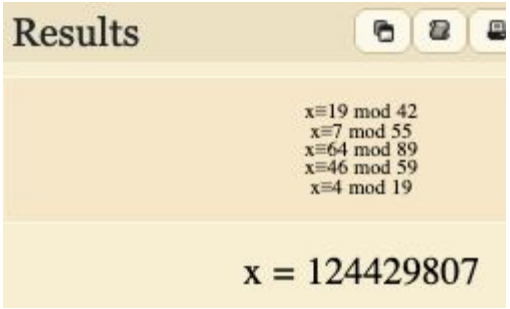
CHINESE REMAINDER

Mathematics › Arithmetics › Chinese Remainder

CHINESE REMAINDER CALCULATOR

★ REMAINDERS A AND MODULOS B (FROM EQUATIONS X = A MOD B)

	Remainders A	Modulos B
1	19	42
2	7	55
3	64	89
4	46	59
5	4	19
6	...	...



Results

x≡19 mod 42
x≡7 mod 55
x≡64 mod 89
x≡46 mod 59
x≡4 mod 19

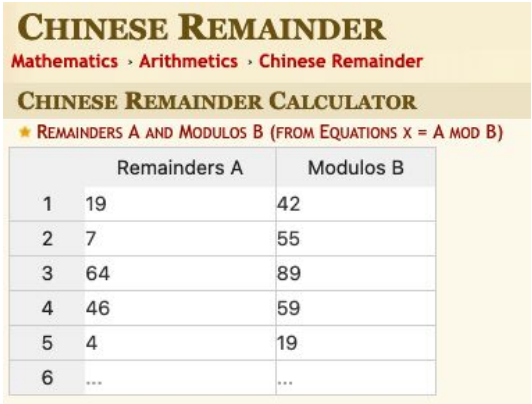
x = 124429807

UNIVERSITÀ DEGLI STUDI
DELL'INSUBRIA

Crypto 10 - CRT

Buondì! Risolvi questo sistema di equazioni modulari per ottenere la flag.

- x % 42 = 19
- x % 55 = 7
- x % 89 = 64
- x % 59 = 46
- x % 19 = 4
- x % 230466390 = ?



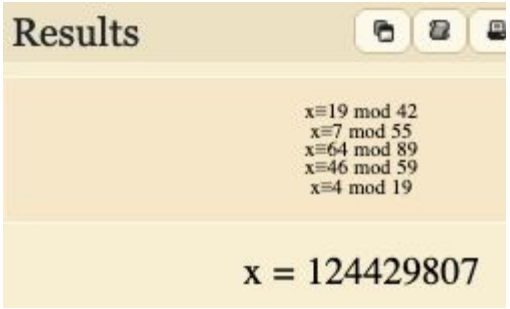
CHINESE REMAINDER

Mathematics › Arithmetics › Chinese Remainder

CHINESE REMAINDER CALCULATOR

★ REMAINDERS A AND MODULOS B (FROM EQUATIONS X = A MOD B)

	Remainders A	Modulos B
1	19	42
2	7	55
3	64	89
4	46	59
5	4	19
6	...	...



Results

x≡19 mod 42
x≡7 mod 55
x≡64 mod 89
x≡46 mod 59
x≡4 mod 19

x = 124429807

- print(124429807 % 230466390)
  - # 124429807

flag:flag{Ch1n3s3_m4th3m4t1c14n5_4r3_D0p3!}

One-way functions

A function F: {0, 1}* -> {0, 1}* is one-way if it is easy to compute and hard to invert

Formally

- $\forall$ x, F(x) is computed by a polynomial time algorithm

Example

- Modular exponentiation is believed to be a one-way function
- x => g^x mod p

Easy and Hard problems

We can distinguish number theory problems into two types

- "Easy" problems: problems that can be solved with efficient algorithms
- "Hard" problems: problems for which efficient algorithms are not known

Some "hard problems" are impossible to solve in practice and allow us to create schemes with "strong" security proofs

Prime factorization and Discrete Logarithms

"Easy" problem

- product of prime numbers
  - $13 * 7 = ?$

"Hard" problem

- ?

"Easy" problem

- modular exponentiation
  - $13^7 \equiv ?$

"Hard" problem

- ?

Prime factorization and Discrete Logarithms

"Easy" problem

- product of prime numbers
  - $13 * 7 = 91$

"Hard" problem

- ?

"Easy" problem

- modular exponentiation
  - $13^7 \equiv ?$

"Hard" problem

- ?

Prime factorization and Discrete Logarithms

"Easy" problem

- product of prime numbers
  - $13 * 7 = 91$

"Hard" problem

- ?

"Easy" problem

- modular exponentiation
  - $13^7 \equiv 9 \bmod 23$

"Hard" problem

- ?

Prime factorization and Discrete Logarithms

"Easy" problem

- product of prime numbers
  - $13 * 7 = 91$

"Hard" problem

- prime number factorization
  - $91 = ? * ?$

"Easy" problem

- modular exponentiation
  - $13^7 \equiv 9 \bmod 23$

"Hard" problem

- ?

Prime factorization and Discrete Logarithms

"Easy" problem

- product of prime numbers
  - $13 * 7 = 91$

"Hard" problem

- prime number factorization
  - $91 = ? * ?$

"Easy" problem

- modular exponentiation
  - $13^7 \equiv 9 \bmod 23$

"Hard" problem

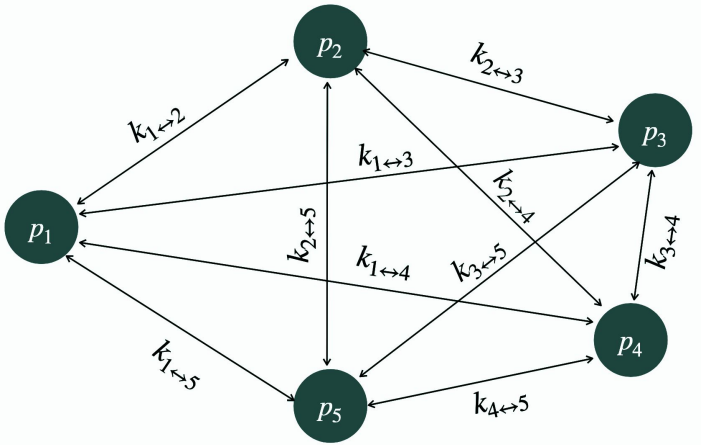
- discrete logarithm
  - $9 \equiv 13^? \bmod 23$

Keys management

Secret keys management

To enable peer communication

- each party has to store n-1 keys



// Scaling up, this becomes a problem: (too) many key pairs

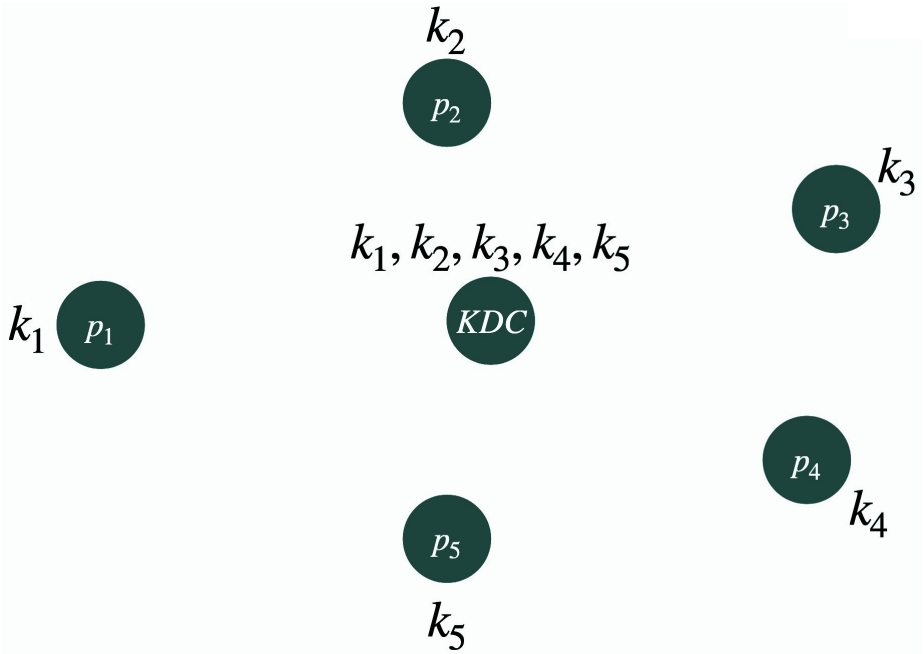
Trusted Third Party

A TTP that stores every secret key of the parties

Problems

- TTP is a single point of failure
- requires a secure channel

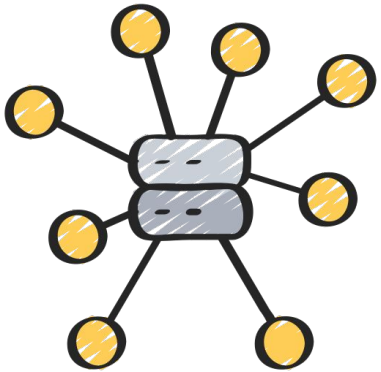
// only usable in controlled environments



Key exchange

How do we transmit keys in a secure way?

- achieve confidential communication without ever communicating over a secure channel
- no shared prior secrets
- adversary is eavesdropping messages, without active actions



So can we do a secure and "online" key exchange?

- Diffie-Hellman

Diffie-Hellman

Diffie-Hellman

- efficient method for exchanging symmetric keys used in many products (TLS 1.3)
- based on the discrete logarithm problem

Two phases

- key generation
- key exchange

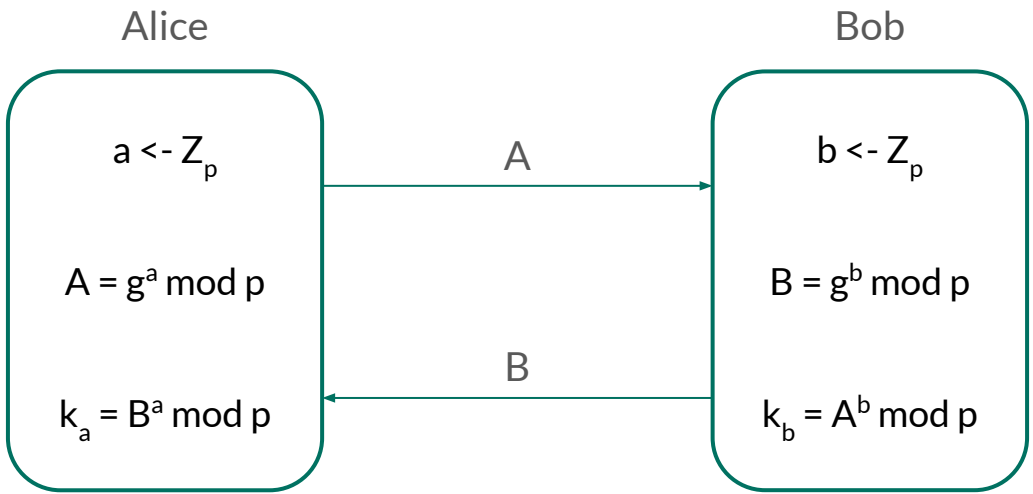
Diffie-Hellman

Public Parameters

- p prime of bit-length n
- g generator of Z_p

$k_a = k_b$?

- $B^a = (g^b)^a = g^{ba} = A^b$



Diffie-Hellman

Example

- Public parameters
  - $p = 37, g = 2$
- Key generation: Alice
  - $a = 7, A = 2^7 \equiv 17 \bmod 37$
- Key generation: Bob
  - $b = 21, 2^{21} \equiv 29 \bmod 37$
- Key exchange
  - $2^{7 \cdot 21} \equiv 29^7 \equiv 17^{21} \equiv 8 \bmod 37$
- Key
  - $k_a = k_b = 8$

Diffie-Hellman

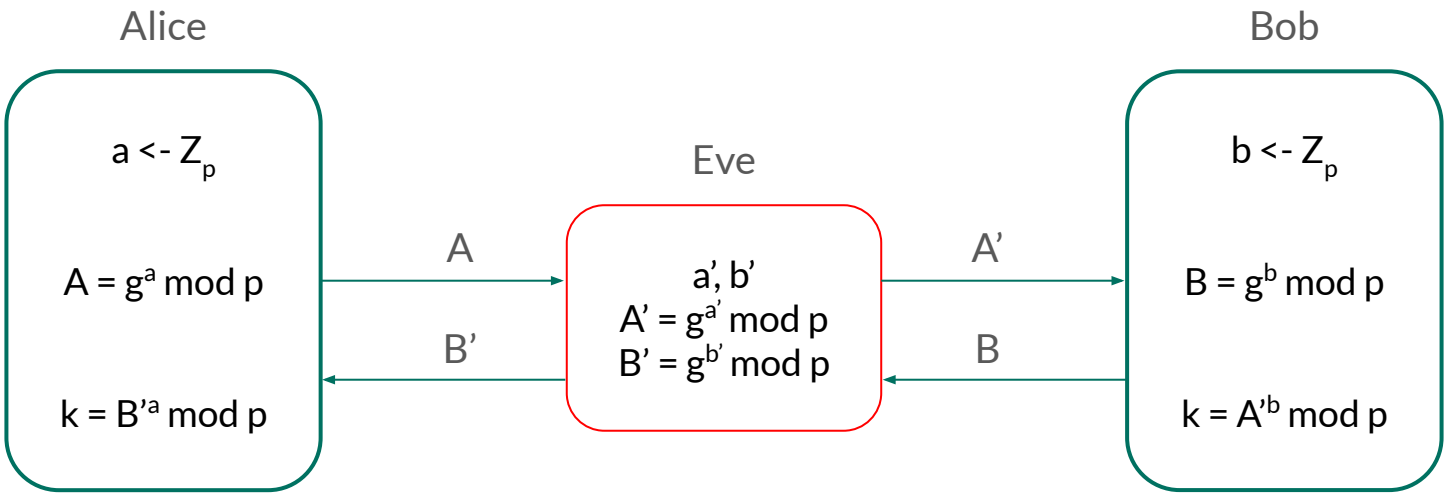
Problems

- vulnerable to Man-In-The-Middle attacks

A listening attacker can

- intercept g^a from Alice and replace it with $g^{a'}$
- intercept g^b from Bob and replace it with $g^{b'}$
- generate keys $g^{a'b}$ e $g^{ab'}$
- decrypt each communication, read it and re-encrypt it with the correct key

Diffie-Hellman Man-In-The-Middle



Challenge

Crypto 12 - DH helpline

Crypto 12 - DH helpline

Codice

```
5   # Se p è un numero primo, quanti numeri positivi minori di p sono coprimi con p? p-1
6
7   # Qual è il logaritmo discreto di 131072 in base 2 (mod 1230346740312797252777902256054304031l9)? 17
8
9   # Qual è il logaritmo discreto di 10 in base 2 (mod 19)? 17
10
11  # Parametri pubblici: p = 191, g = 2.
12  # La mia chiave pubblica è 149.
13  print(pow(2, 11, 191))
14
15  # Qual è la tua chiave pubblica? 138
16  print(pow(149, 11, 191))
17
18  # Grazie! Qual è la nostra chiave condivisa? 156
19
```

Crypto 12 - DH helpline

flag: flag{W3_st4Nd_t0d4Y_0n_th3_Br1nk_of_4_r3v01uTioN_1n_Cryptography.}

Hint!
Check the difference between prime numbers and "safe" prime numbers

Challenge

Crypto 13 - A Diffiecult communication

Crypto 13 - A Diffiecult communication

```python
from Crypto.Random import get_random_bytes
from Crypto.Util.number import getPrime, isPrime

while True:
    p = getPrime(1024)
    if isPrime((p - 1) // 2, randfunc=get_random_bytes):
        print(p)
        break

p = 151487445871043527820872099207885408..
g = 2
```

Crypto 13 - A Diffiecult communication

```python
from Crypto.Util.number import getRandomInteger

a = getRandomInteger(1024)
A = pow(g, a, p)

a = 14187215684349164492510399160648 92..
A = 26687712321338746656251859298196 59..

B = "f4030f477ba2b2b7215cd28eb83b671f0.."
k = hex(pow(int(B, 16), a, p))[2:]
```

Crypto 13 - A Diffiecult communication

```python
from Crypto.Cipher import AES

IV = "82700697cd1d66fb0134d0748ab08e1b"
msg = "797b771c3f9ff47d813bde3c768dafd.."

IV = bytes.fromhex(IV)
k = bytes.fromhex(k)[:16]
msg = bytes.fromhex(msg)

cipher = AES.new(k, AES.MODE_CBC, iv=IV)
plaintext = cipher.decrypt(msg)
print("Plaintext: " + str(plaintext))
```

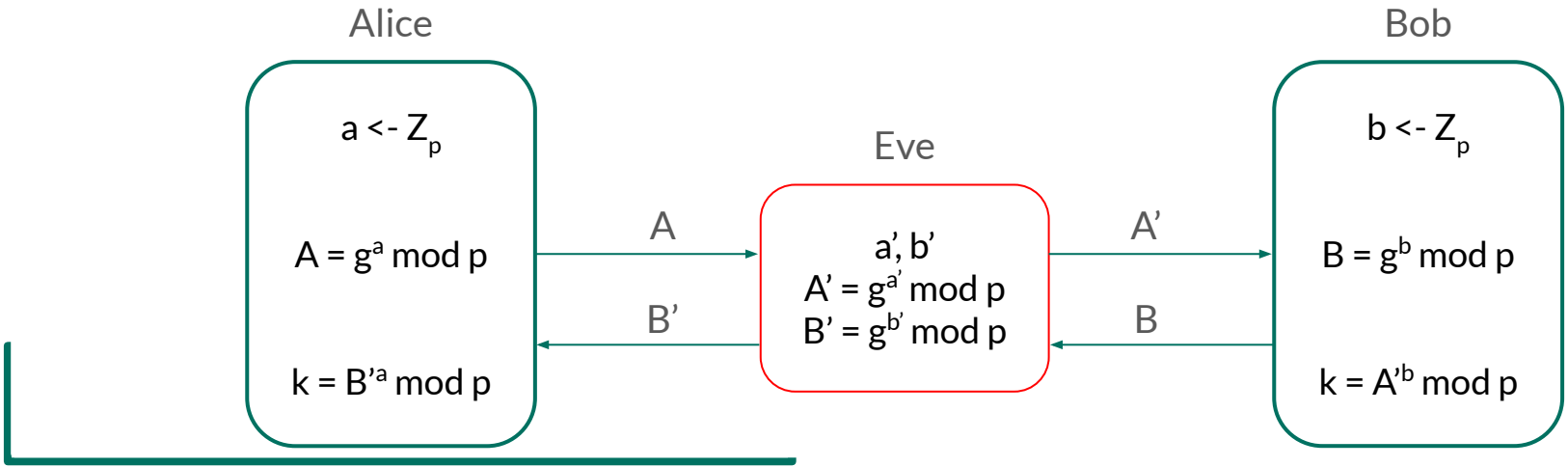
Crypto 13 - A Diffiecult communication

flag: flag{D1_qU3st1_73mPi_CrYP70_3_+_1mp0R74Nte_cH3_m4i}

Diffie-Hellman

Solution?

- public key signature schemes proving that a given key corresponds to a given entity



Asymmetric Encryption

Asymmetric Encryption

Currently, Alice and Bob

- agree on a cipher to use
- exchange a key
- begin to communicate using the cipher and key

Symmetric encryption issues

- key distribution
  - large number of keys to manage
  - secure key distribution protocols
- digital signature
  - method to guarantee that a message was actually produced by the sender

Asymmetric Encryption

Two keys are used

- **public**: can be known by everyone, used to encrypt messages and verify signatures
- **private**: known only by the owner, used to decrypt messages and generate signatures

Applications

- **encryption**: Encryption of a message with the recipient's public key
- **digital signature**: a sender signs the message with his private key
- **key exchange**: two users exchange a symmetric key

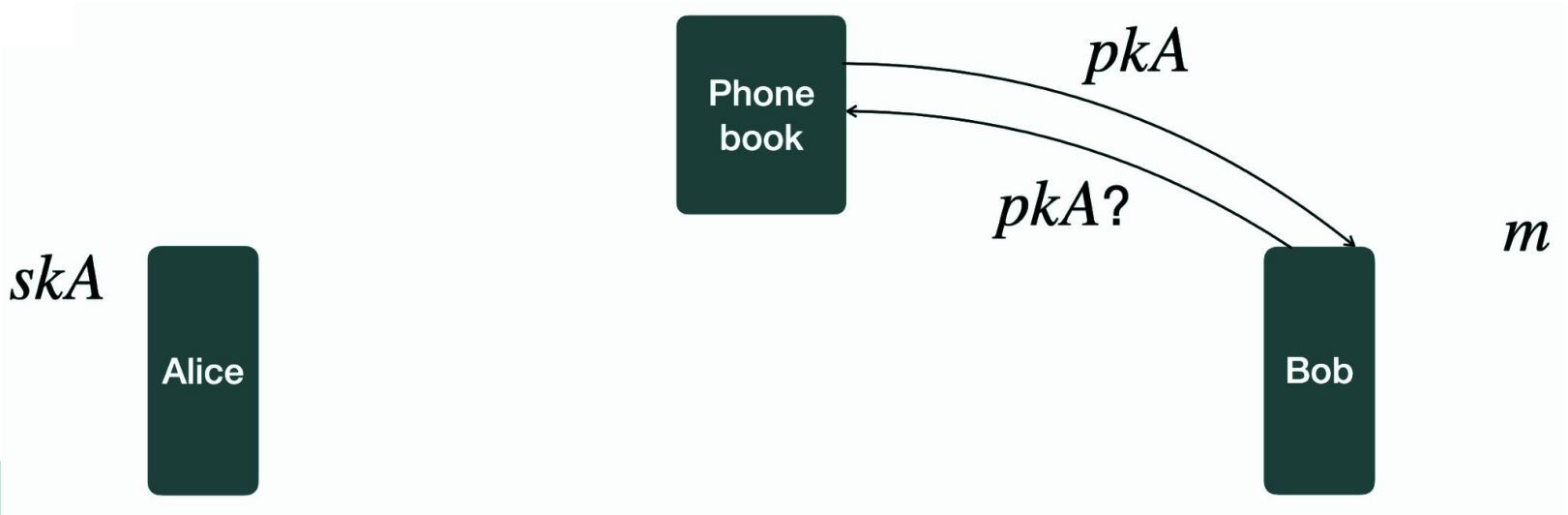
Asymmetric Encryption

A key is a pair of values (pk, sk) linked together in some way. Users share pk publicly (public key) and keep sk secret (secret key).

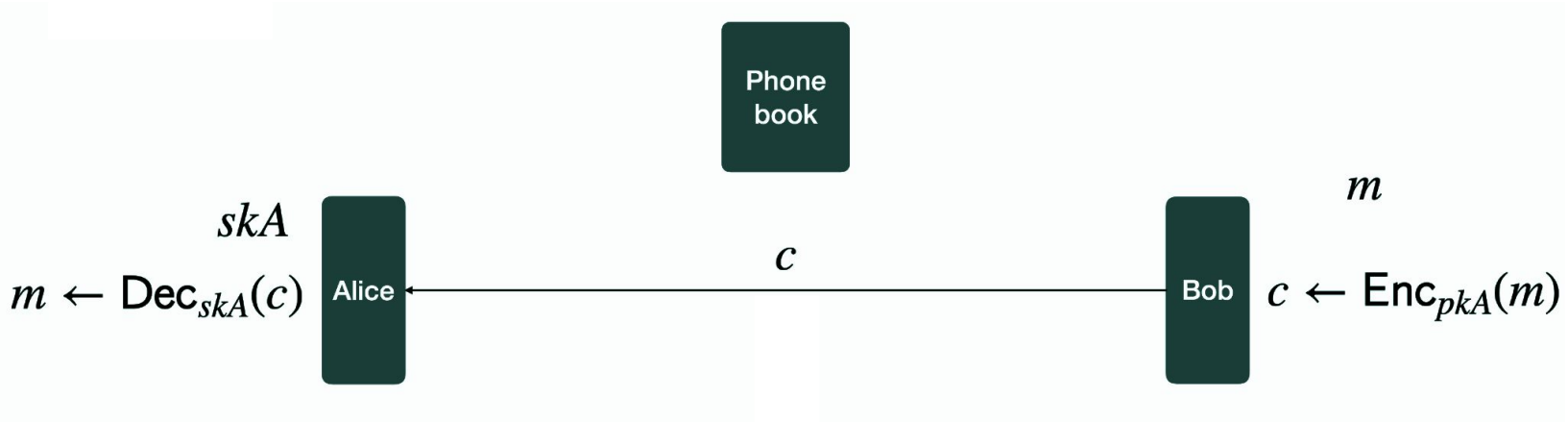
Example

- Bob wants to send a message to Alice
  - Bob takes Alice's public key pk_{Alice}
  - Bob encrypts the message as c = $E(pk_{Alice}, m)$ e lo invia
  - Alice can decrypt the message as $D(sk_{Alice}, c)$
  - No one else can decrypt the message without knowing the value of sk_{Alice}

Asymmetric Encryption



Asymmetric Encryption



Asymmetric Encryption

Formal definition: triple of polynomial time algorithms

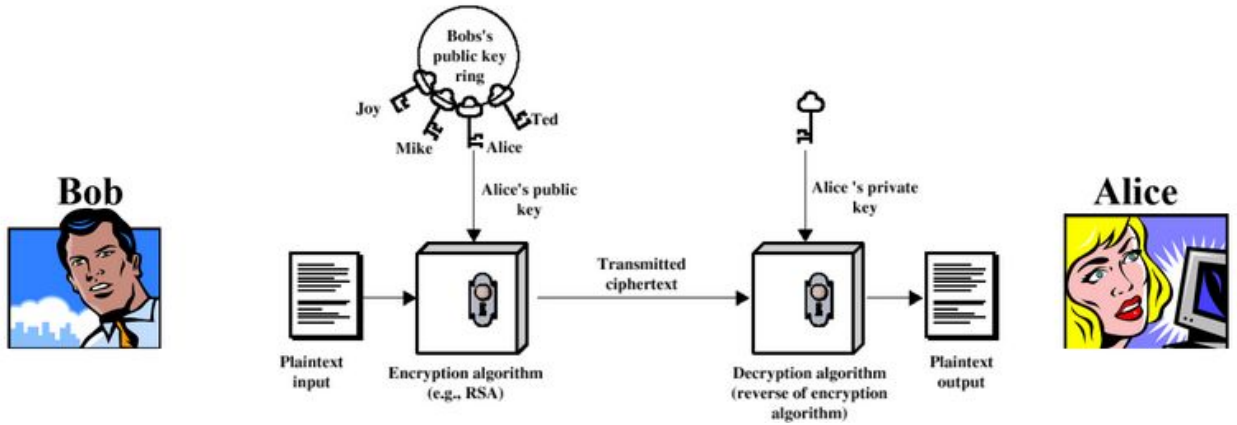
- (pk, sk) <- Gen(n)
  - n is the key size and the message space is determined by the public key
- c <- Enc(pk, m)
  - the message m is from a proper message space
- m <- Dec(sk, c)

// correctness: $Dec_{sk}(Enc_{pk}(m)) = m$

Asymmetric Encryption

Confidentiality

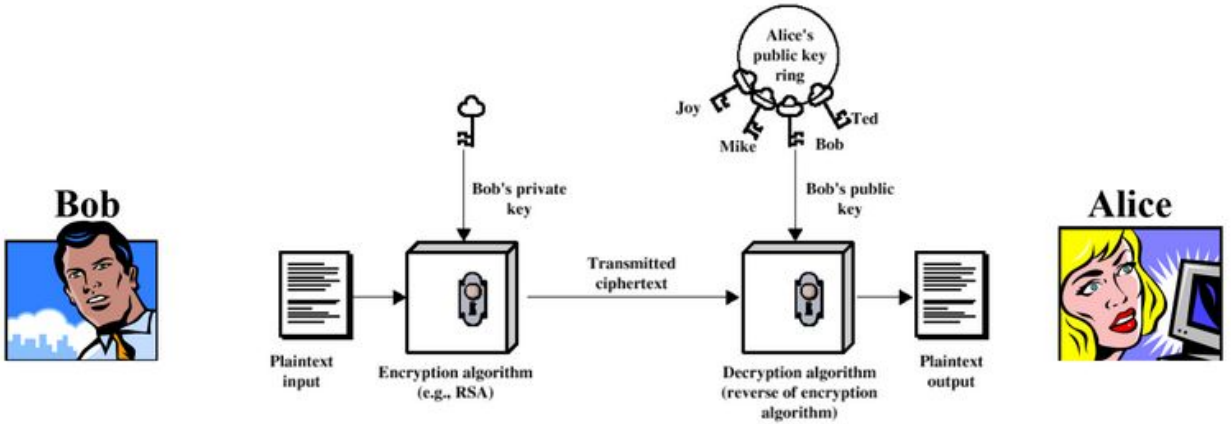
- Bob wants to send a confidential message to Alice



Asymmetric Encryption

Authentication

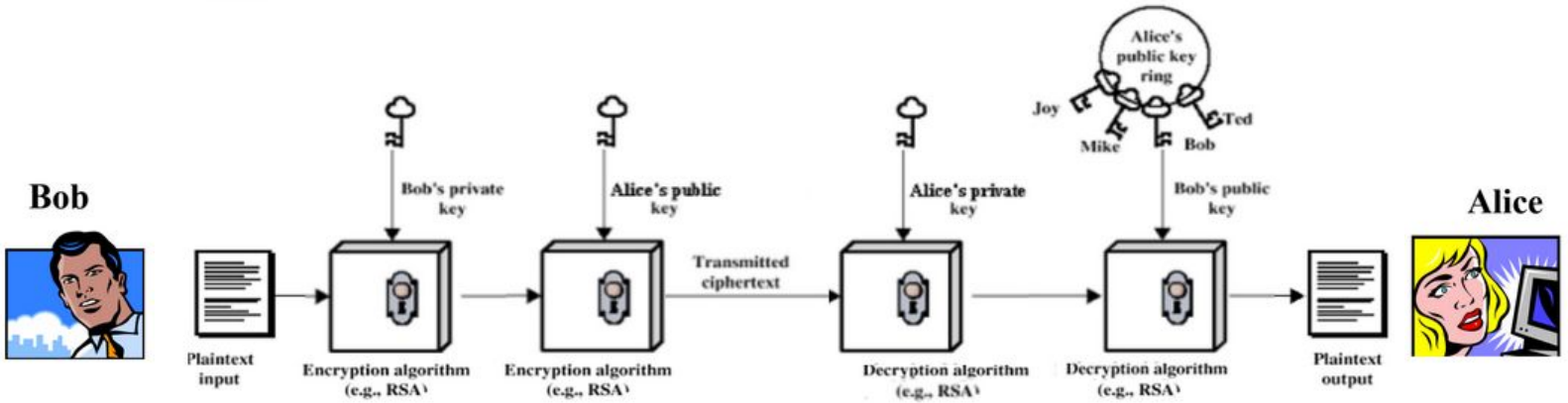
- Bob wants to send a message that can be proven to be authentic



Asymmetric Encryption

Confidentiality and Authentication

- Bob wants to send a secret and authenticable message to Alice



Asymmetric Encryption

Asymmetric encryption complements symmetric encryption rather than being an alternative to it

Asymmetric Encryption

Public Key Encryption schemes

- Discrete Logarithm Problem
  - El Gamal
  - Diffie-Hellman (key exchange)
  - DSA
  - Schnorr (digital signature)

- Integer Factorization Problem
  - RSA, Rabin, Paillier

Integer Factorization Problem

Integer Factorization Problem

- find prime factors of a given large composite number
- given a number N, find two (or more) smaller numbers that multiply together to give N

Formally

- Every integer N > 1 can be written as $N = \prod p_i^{ei}$
  - p_i are distinct primes, e_i are integers >=1

// factorisation is hard for certain types of composites (hardest if factors are only large primes)

One-way functions

A function F: {0, 1}* -> {0, 1}* is one-way if it iseasy to compute and hard to invert

Formally

- ∀ x, F(x) is computed by a polynomial time algorithm

Example

- Integer Factoring Problem is believed to be a one-way function
- primes p, q => N = p * q

RSA - Rivest, Shamir e Adleman

Most famous and currently used asymmetric encryption scheme

- uses large integers (typically 2048 bits)
- **encryption**: based on the operation of raising to a power of integers modulo prime numbers (simple operation)
- **decryption**: security is guaranteed by the cost of factoring large numbers (difficult operation)

RSA - Rivest, Shamir e Adleman

common values of *e* are 3 (too small - not secure) and 65537

Formal definition: triple

- (pk, sk) <- Gen(n)
  - pick two random n-bit primes p, q
  - compute N = p * q
  - pick integer e s.t. gcd(e, (p-1) * (q-1)) = 1
  - computes d <- e^{-1} mod (p-1) * (q-1)
  - output (pk, sk) <- ((e, N), d)
- c <- Enc(pk, m)
  - for m $\in Z^*_N$ and pk = (e, N), compute c <- m^e mod N
- m <- Dec(sk, c)
  - for c $\in Z^*_N$ and sk = d, compute m <- c^d mod N

RSA - Rivest, Shamir e Adleman

Example

- key generation
  - Pick p = 11 and q = 17 ⇒ N = 187 e $\varphi(N)$ = 160
  - Pick e = 3 and compute $d \equiv 3^{-1} \equiv 107 \bmod \varphi(N)$
- encryption
  - Pick m = 10 and compute $c \equiv 10^3 \equiv 65 \bmod N$
- decryption
  - $c^d \equiv 65^{107} \equiv 10 \bmod N$

RSA - Rivest, Shamir e Adleman

```python
def private_key(n):
    p = getPrimeOver(n)
    q = getPrimeOver(n)
    return p, q


def public_key(p, q, e=65537):
    N = p * q
    return N, e


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


def decrypt(ciphertext, p, q, N, e):
    totient = N - (p + q) + 1
    d = pow(e, -1, totient)
    return pow(ciphertext, d, N)
```

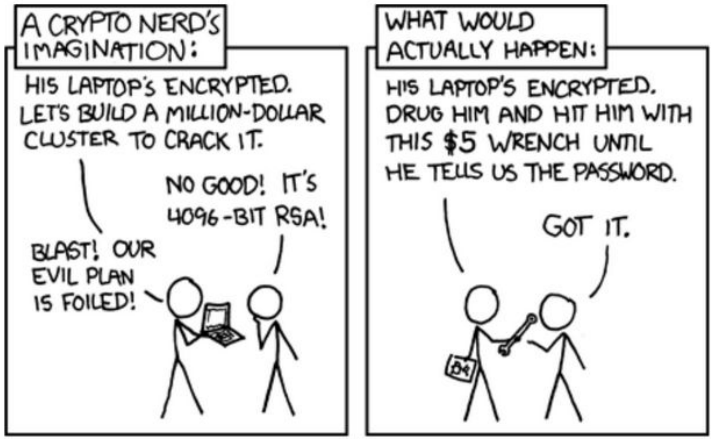
RSA - Rivest, Shamir e Adleman

Security depends on the choice of key

- key
  - it is reasonable to use keys of 2048 bits
  - but the longer the key, the more expensive the process
- e
  - 3 is not secure
  - 65537 = $2^{16} + 1$

How to choose the length of the key?

- there are several recommendations
  - NIST
  - NSA
  - RFC 3766
  - ECRYPT



Challenge

Crypto 11 - RSA helpline

Crypto 11 - RSA helpline

```python
p = 13
q = 5

N = p * q

plaintext = 8

f_n = (p - 1) * (q - 1)

e = 5

ciphertext = pow(plaintext, e, N)

d = 29

assert pow(ciphertext, d, N) == plaintext
```

flag: flag{ugh_NumB3r5_41r34dy}

RSA Attacks

Based on

- modulus
- oracles
- small public exponents
- small private exponents

RSA Attacks: modulus

Modulo Factorization Attacks

- in general, no chance for numbers larger than 1048 bits

Possible if

- primes are reused
- at least one prime is small
- two primes are very close

RSA Attacks: modulus

One of the most popular algorithms is the General Number Field Sieve (GNFS)

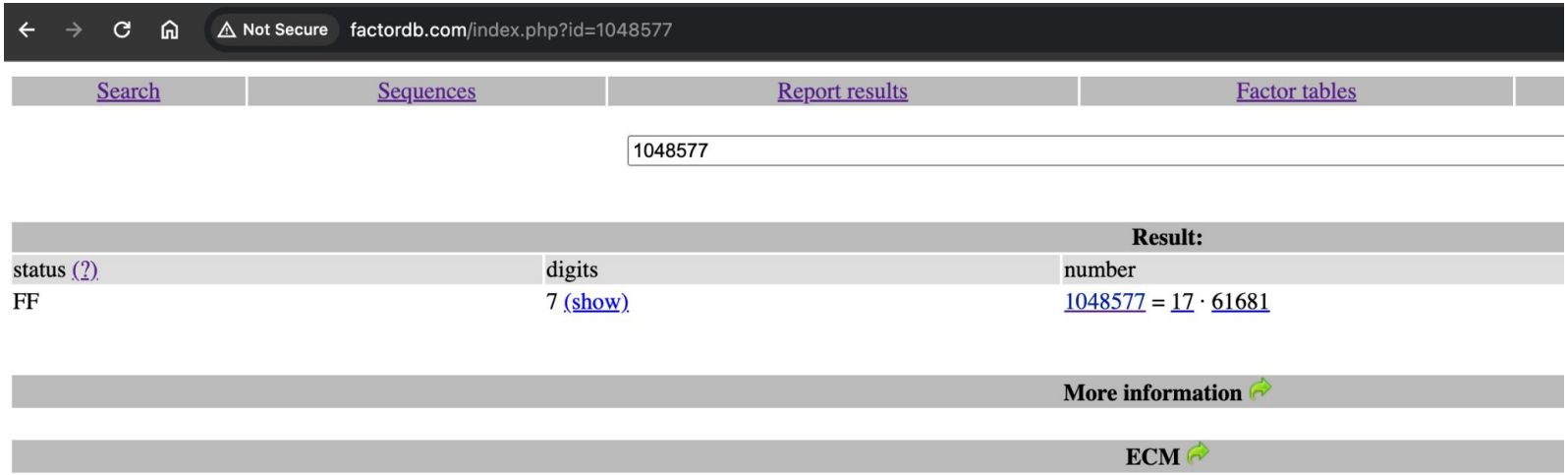
- tool: CADO-NFS[1]

Alternative

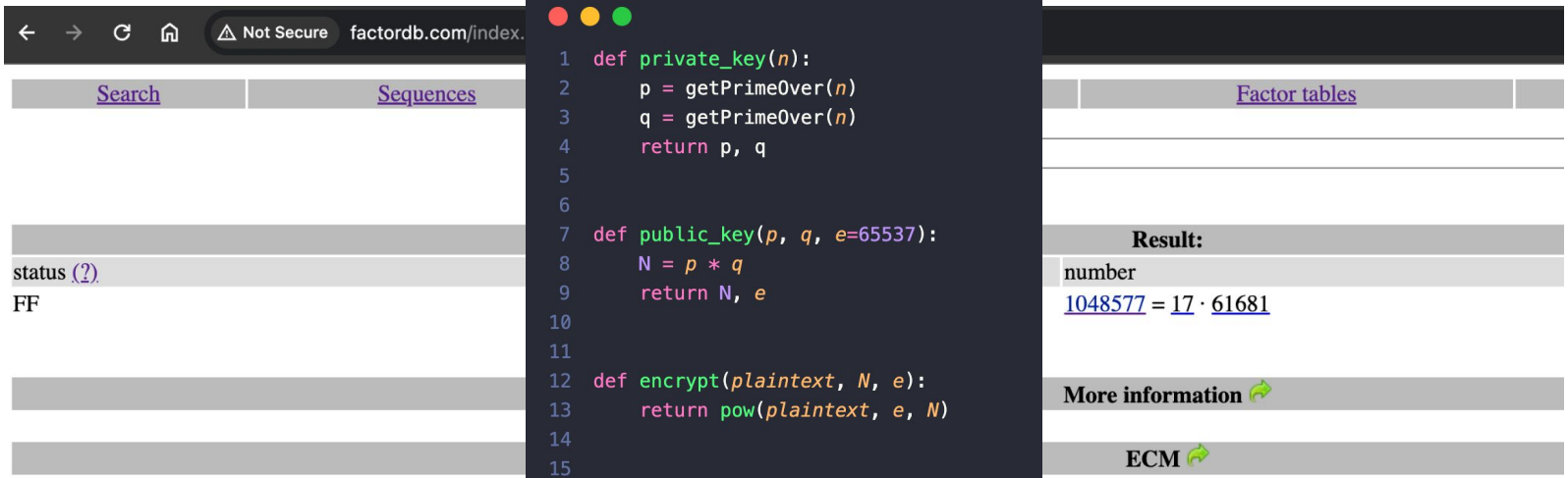
- factorDB[1]
- YAFU[2]

[1] http://factordb.com/
[2] https://github.com/DarkenCode/yafu

RSA Attacks: modulus



RSA Attacks: modulus




```python
def private_key(n):
    p = getPrimeOver(n)
    q = getPrimeOver(n)
    return p, q


def public_key(p, q, e=65537):
    N = p * q
    return N, e


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


def decrypt(ciphertext, p, q, N, e):
    totient = N - (p + q) + 1
    d = pow(e, -1, totient)
    return pow(ciphertext, d, N)
```

Fermat's squares difference attack

Special case

- let $p = a + b$ e $q = a - b$ where a, b are two integers
- the RSA module becomes $N = a^2 - b^2$ -> $a^2 - N = b^2$

If b is sufficiently small, it is possible to perform a brute-force attack on a or b to obtain the factorization

Common Prime Attack

It has been shown that in real use cases it is possible for two RSA keys to share one of the prime factors

Why could this be a problem?

```
1  p = 2147990771
2  q = 3248916997
3  r = 1058313049
```

Given $N_1 = p \times q$ e $N_2 = p \times r$, it is possible to factorize the two modules as $p = GCD(N_1, N_2)$

- Euclid's algorithm runs in polynomial time
- obtained p, the two RSA keys are broken

Common Prime Attack

```
1  from math import gcd
2
3  p = 2147990771
4  q = 3248916997
5  r = 1058313049
```

Common Prime Attack

```python
from math import gcd

p = 2147990771
q = 3248916997
r = 1058313049

N1 = p * q
N2 = p * r
print(N1)  # 6978643725301034687
print(N2)  # 2273246662080870779

p = gcd(N1, N2)
print(p)   # 2147990771
```

Common Modulus Attack

Suppose we have two keys (N, e_1), (N, e_2) with $GCD(e_1, e_2) = 1$ and two ciphertext c_1, c_2 coming from the same plaintext

- using the Bézout's identity, we can compute *u* and *v* such that $u \times e_1 + v \times e_2 = 1$
- computing $(c_1{}^u \bmod N) * (c_2{}^v \bmod N) \bmod N$ we can get the plaintext!

```
N = 6978643725301034687
e1 = 65537
e2 = 3


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)
```

Common Modulus Attack

```python
from binascii import hexlify, unhexlify

N = 6978643725301034687
e1 = 65537
e2 = 3


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


plaintext = "secret"
plaintext = int(hexlify(plaintext.encode()), 16)

..

plaintext = unhexlify(hex(t)[2:].encode())
print(plaintext.decode())

```

Common Modulus Attack

```python
from binascii import hexlify, unhexlify

N = 6978643725301034687
e1 = 65537
e2 = 3


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


plaintext = "secret"
plaintext = int(hexlify(plaintext.encode()), 16)

..
```

```python
..

c1 = encrypt(plaintext, N, e1)
c2 = encrypt(plaintext, N, e2)

u = -1
v = 21846

t1 = pow(c1, u, N)
t2 = pow(c2, v, N)

t = t1 * t2 % N

plaintext = unhexlify(hex(t)[2:].encode())
print(plaintext.decode())  # secret
```

Oracles

Types of oracles

- Encryption oracles
- Decryption oracles
- Padding oracles

Modulus Recovery

Scenario

- we have a cryptographic oracle that encrypts with a fixed secret key

Can we recover the modulus??

- encrypt two chosen messages x and y
- we use the fact that the standard RSA exponent is 65537 to compute x^e e y^e (without modular reduction)
- we note that $x^e - Enc(x)$ is a multiple of N
- compute $GCD(x^e - Enc(x), y^e - Enc(y))$ to find small multiples of N
- repeat with different values if necessary

Modulus Recovery

```python
from binascii import hexlify
from math import gcd

N = 6978643725301034687
e = 65537


def encrypt(plaintext, N=N, e=e):
    return pow(plaintext, e, N)


p_1 = int(hexlify(b"secret1"), 16)
p_2 = int(hexlify(b"secret2"), 16)

..
```

Modulus Recovery

```python
from binascii import hexlify
from math import gcd

N = 6978643725301034687
e = 65537


def encrypt(plaintext, N=N, e=e):
    return pow(plaintext, e, N)


p_1 = int(hexlify(b"secret1"), 16)
p_2 = int(hexlify(b"secret2"), 16)

..
```

```python
..

c1 = encrypt(p_1)
c2 = encrypt(p_2)

t1 = pow(p_1, e)
t2 = pow(p_2, e)

mod = gcd(t1 - c1, t2 - c2)
print(mod)  # 6978643725301034687
```

Homomorphic properties of RSA

Scenario

- we have a decryption oracle that does not allow us to decrypt messages containing sensitive data

undecipherable secret?

Can we decrypt arbitrary ciphertexts?

- we encrypt a chosen plaintext x (locally!)
- we ask the server to decrypt $x^e * c$
- we note that $D(x^e * c) = D(x^e) * D(c) = x * D(x)$ and recover the message

Homomorphic properties of RSA

```python
from binascii import hexlify, unhexlify

N = 6978643725301034687
e = 65537

p = 2147990771
q = 3248916997


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


def decrypt(ciphertext, p, q, N, e):
    totient = N - (p + q) + 1
    d = pow(e, -1, totient)
    return pow(ciphertext, d, N)

..
```

Homomorphic properties of RSA

```python
from binascii import hexlify, unhexlify

N = 6978643725301034687
e = 65537

p = 2147990771
q = 3248916997


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


def decrypt(ciphertext, p, q, N, e):
    totient = N - (p + q) + 1
    d = pow(e, -1, totient)
    return pow(ciphertext, d, N)

..
```

```python
..

x = int(hexlify(b"pws"), 16)
c = encrypt(x, N, e)

t = pow(x, e) * c
d = decrypt(t, p, q, N, e)
res = d // x

res = unhexlify(hex(res)[2:].encode())
print(res.decode())  # pws
```

LSB Oracle

Scenario

- we have a decryption oracle that provides a partial decryption: only the least significant bit

Can we recover the plaintext?

- The idea is to find the answer using the fact that N is odd
- let's decipher $2^e * c$: if the result is 1, then $2m > N$ and therefore $N/2 < m < N$, otherwise $0 < m < N/2$
- let's repeat with $4^e * c$, $8^e * c$ and so on

LSB Oracle

```python
from binascii import hexlify, unhexlify
import numpy as np

N = 6978643725301034687
e = 65537

p = 2147990771
q = 3248916997


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


def decrypt(ciphertext, p, q, N, e):
    totient = N - (p + q) + 1
    d = pow(e, -1, totient)
    r = pow(ciphertext, d, N)
    r_array = bytearray(r.to_bytes(20, "big"))
    return np.unpackbits(r_array)[-1]
```

```python
plaintext = int(hexlify(b"secret"), 16)
c = encrypt(plaintext, N, e)

..
```

LSB Oracle

```python
from binascii import hexlify, unhexlify
import numpy as np

N = 6978643725301034687
e = 65537

p = 2147990771
q = 3248916997


def encrypt(plaintext, N, e):
    return pow(plaintext, e, N)


def decrypt(ciphertext, p, q, N, e):
    totient = N - (p + q) + 1
    d = pow(e, -1, totient)
    r = pow(ciphertext, d, N)
    r_array = bytearray(r.to_bytes(20, "big"))
    return np.unpackbits(r_array)[-1]
```

```python
plaintext = int(hexlify(b"secret"), 16)
c = encrypt(plaintext, N, e)

lower_limit = 0
upper_limit = N

i = 1

while lower_limit < upper_limit:
    t = pow(2**i, e) * c
    t_d = decrypt(t, p, q, N, e)
    if t_d == 0:
        upper_limit = (upper_limit + lower_limit) / 2
    else:
        lower_limit = (lower_limit + upper_limit) / 2
    i += 1

res = unhexlify(hex(int(lower_limit))[2:].encode())
print(res.decode())  # secret
```

Padding oracles

Most famous attacks

- Coppermith's attack against short padding schemes
- Bleichenbacher's attack against PKCS#1 v1.5
- Manger's attack against PKCS#1 v2.0

Attacks on small public exponent

Direct Cube Root - Scenario

- We have a padding-free implementation of RSA with e = 3 and an encrypted message that comes from a "short message"

Can we decipher it?

- assumption: if the number of bits in the message is less than $\log_2 N/3$, the module has no effect

By calculating the cube root we find the message

- encryption: $c = p^e \pmod n$ -> attack: $p = \sqrt[3]{c}$ (if $p^e < n$)

Attacks on small public exponent

c =
000608a5a8b32e3b218b49f9f0c33d05f135eff53da
bee5ac00ec13e72997ef7c4b478d6b95cecc01ebcf7
cecf24785ba28b4b88f605ee93d6502f24635624ccf
4772e0d7334d125e047853c4a42e21d4bc3412729
b0d230c1f5d6a2f8272bb18927313328c6e431bf1e
38d5200a5ac32293f4d58c467636a20bc7393d7133
c365880f07a2a29ad63fb21ebd2011b02dcd298fbd
97c70d72fbc5c433a223987d898da302aa100efd95f
43e7d6698a7eeea05dd97870583f149beda3aa6334
ba4641c224e8c8f2cd61156abd42e4e3d03070c453
656ec072526618e2ad74c87e89c54397de803b8e2
347f7daf413695e1df816782758f26a2abfe947c82c
e9e9d

```python
from binascii import hexlify, unhexlify
import gmpy2

c = "000608a5a8b32e3b218b49f9f0c33d05f1.."
c = int(c, 16)

e = 3
```

Attacks on small public exponent

```python
from binascii import hexlify, unhexlify
import gmpy2

c = "000608a5a8b32e3b218b49f9f0c33d05f1.."
c = int(c, 16)

e = 3

nth_root = gmpy2.iroot(c, e)[0]
m = unhexlify(hex(nth_root)[2:].encode())
print(m.decode())
 # If you use padding and a sensible exponent (e.g., e=65537), then RSA should be secure
```

Attacks on small public exponent

Hastad's broadcast Attack - Scenario

- three entities have 3 distinct RSA keys (N_1, 3), (N_2, 3), (N_3, 3)
- the same message is sent to entities without padding

Knowing the 3 ciphertexts, can we decipher them?

- using the Chinese Remainder Theorem we can get
  - $m^3 \, mod \, N_1 \times N_2 \times N_3$
- since m < min($N1$, $N2$, $N3$) it is possible to use the Direct Cube Root

Attacks on small public exponent

Hastad's broadcast Attack - Scenario

- three entities have 3 distinct RSA keys (N_1, 3), (N_2, 3), (N_3, 3)
- the same message is sent to entities without padding

Knowing the 3 ciphertexts, can we decipher them?

- using the Chinese Remainder Theorem we can get
  - $m^3 \, mod \, N_1 \times N_2 \times N_3$
- since m < min($N1$, $N2$, $N3$) it is possible to use the Direct Cube Root

Challenges

CR_2.06 - Small key
CR_2.07 - Big key
CR_2.08 - Evil Carol 1 / 2

Challenges

CR_2.01 - RSA-Walkthrough
CR_2.02 - Decrypt-me 1
CR_2.03 - Decrypt-me 2
CR_2.04 - RS2A
CR_2.05 - Oracle - Level 1 /2 /3/ 4
CR_2.09 - lonely primes
CR_2.10 - Friendly primes
CR_2.11 - PaaS
CR_2.12 - Shell game
CR_2.13 - private prime
CR_2.14 - notcoprime
CR_2.15 - maybehard